CBSA: Content-based Soft Annotation for Multimodal Image Retrieval Using Bayes Point Machines

Edward Chang
Electrical & Computer Engineering
University of California
echang@ece.ucsb.edu

Kingshy Goh
Electrical & Computer Engineering
University of California
kingshy@engineering.ucsb.edu

Gerard Sychay
Computer Science
University of California
gerard@cs.ucsb.edu

Gang Wu
Electrical & Computer Engineering
University of California
gwu@engineering.ucsb.edu

Abstract

We propose a content-based soft annotation (CBSA) procedure for providing images with semantical labels. The annotation procedure starts with labeling a small set of training images, each with one single semantical label (e.g., forest, animal, or sky). An ensemble of binary classifiers is then trained for predicting label membership for images. The trained ensemble is applied to each individual image to give the image multiple soft labels, and each label is associated with a label membership factor. To select a base binary-classifier for CBSA, we experiment with two learning methods, Support Vector Machines (SVMs) and Bayes Point Machines (BPMs, and compare their class-prediction accuracy. Our empirical study on a 116-category 25K image set shows that the BPM-based ensemble provides better annotation quality than the SVM-based ensemble for supporting multimodal image retrievals.

Keywords: Bayes Point Machines, Support Vector Machines, image annotation, multimodal image retrieval.

1 Introduction

Content-based image retrieval supports image searches based on perceptual features, such as color, texture, and shape. However, for most users, articulating a content-based query using these low-level features can be non-intuitive and difficult. Many users prefer using keywords to conduct searches. We believe that a combined keyword- and content-based approach can benefit from the strengths of both paradigms. A user can start a query by entering a few keywords. Once a few images relevant to the query are found, the image system uses these images' perceptual features, together with their annotation, to perform a multimodal query refinement.

Images must first be annotated in order to support such combined keyword- and content-based queries and refinement. In this work we propose a content-based soft annotation (CBSA) approach to provide images with multiple semantical labels. The initial input to CBSA is a training image set in which each image has been manually annotated with a single semantical label. The mission of CBSA is to propagate these labels to unlabeled images as well as to the labeled ones. At the end of the annotation process, each image is annotated with a label-vector, and each label in the vector is assigned a confidence factor. For instance, each image in a training set is initially labeled with one of K labels such as *forest*, *tiger*, *sky*, and so on. At the end of the CBSA process, each image is annotated with a label-vector of K labels. An image label-vector $(forest : 0.1, tiger : 0.9, sky : 0.7, \cdots)$ means that the image is believed to contain semantics of *forest*, *tiger*, and *sky* with 10%, 90%, and 70% confidence, respectively. When a text-based search is issued with keywords, images are ranked and retrieved based on their combined confidence factors in the matching labels[1].

As pointed out by [3, 4], automatic annotation may not attain extremely high accuracy with the present state of the computer vision and image processing technology. However, providing images with some reliable initial semantical labels and then refining these *unconfirmed* labels via relevance feedback is believed to be an effective approach [44, 52]. (We discuss some refinement methods that can improve annotation accuracy in Section 2.) CBSA aims to initialize images with a set of semantical words whose reliability is significantly better than chance. Our empirical study shows that even though the initial annotation may not be perfect, CBSA assists a user to find some relevant images rapidly via a keyword search. Once some relevant images are found, a query refinement method such as MEGA [27] and SVM_{Active} [46] can be employed to zoom quickly into the user's query concept.

More specifically, CBSA consists of three steps. We summarize these steps as follows and highlight our contributions:

1. Manually labeling each training images with one of the K pre-selected semantic labels. Our labeling scheme is simple since it does not involve segmenting of images [3, 4], or manual annotating of an image with multiple keywords and probabilities [32].

2. Train K classifiers. Using the labeled instances, we train an ensemble of K binary classifiers (using Bayes Point Machines (BPMs) or Support Vector Machines(SVMs)). Each classifier assumes the task of determining the confidence score for a semantic label. Our K-class ensemble automatically adapts kernel parameters (i.e., C and γ of the Laplacian kernel) for each of the K class-boundaries based on the idiosyncrasies of that individual class. Our empirical study shows that employing BPMs as the base classifier outperforms SVMs [48]

[1] When a query keyword is not in the label set or it has multiple senses, query refinement/expansion is needed to translate the initial query. Detailed treatment on query refinement/expansion is beyond the scope of this study.

in class-prediction accuracy. We also provide theoretical justification for BPMs' superiority.

3. Automatically annotating images using the classifiers. Each image is classified by the K classifiers and assigned a confidence score for the label that each classifier is attempting to predict. As a result, a K-nary label-vector consisting of K-class membership is generated for each image.

The rest of the paper is organized into five sections. In Section 2, we discuss related work, explaining how our approach differs from others. In Section 3, we present the SVMs and BPMs one-per-class (OPC) ensemble schemes that we used for annotating images. We also present a kernel parameters selection method, which tends to minimize generalization errors. Section 4 presents our CBSA annotation method. In Section 5, we present and discuss our experimental results. Finally, we offer our concluding remarks in Section 6, together with some ideas for future research.

2 Related Work

We discuss related work in classification ensembles and annotation.

2.1 Related Work in Classification Ensembles

The regression approach to classification considers the discrimination problem of K classes simultaneously. This method is computationally expensive ($O(K^3)$) and may impose a universal assumption on class boundaries [15]. A more flexible and cheaper ($O(K^2)$) approach is the use of ensemble schemes.

Ensemble methods can be divided into two categories based on their design goals. The goal of the first category is to reduce prediction variance caused by training data selection. Some well-known methods in this category include bagging [6], arcing [7], and boosting [41].

The goal of the second ensemble category is to reduce prediction error using decomposition and reconstruction methods with good error-correction capability [2, 14, 47]. Three representative schemes are:

1. One Per Class (OPC). This scheme trains K binary classifiers for K classes. The classifier which gives the largest output determines the class of a given data point.

2. Pairwise Coupling (PWC). This method trains $\frac{1}{2}K(K-1)$ binary classifiers, each of which provides a partial decision for classifying a data point. PWC then combines the output of

all classifiers to form a class prediction. The study of [31] shows that combining these decision outputs differently may yield different class decisions.

3. Error-Correcting Output Coding (ECOC). ECOC was first proposed by Dietterich and Bakiri [14] to reduce classification error by exploiting the redundancy of coding schemes. Each class is assigned a codeword of length L. The value of L has to be much larger than the number of classes K. This allows each pair of codewords to have a large Hamming distance, and hence good error correction. To make such pairing possible, the codewords must be designed to have good row and column separation. For large K, an optimal set of codewords cannot be generated deterministically.

Our previous study [16] of the three schemes listed above shows that a simple scheme like OPC performs as well as more elaborate schemes like PWC and ECOC when applied to image classification. Because of OPC's effectiveness and its substantially small number of base classifiers and hence training efficiency, we employ OPC ensemble for combining our binary classifiers.

2.2 Related Work in Annotation

Most multimedia systems that employ semantical information for retrievals are lexicon-based, requiring a framework to extract and represent the semantics of a multimedia object. The provision of an intuitive interface for users is also important. Oftentimes, the user will provide an example that represents his/her query concept, or the system will present some initial examples for the user to choose. This interaction with the users can also further refine the existing annotations. A major challenge faced by such systems is that of propagating semantical annotations from a small number of labeled data to a large number of unlabeled data.

2.2.1 Semantics Extraction

Semantical information is mainly extracted in two ways:

1. Image-based Methods. Image segmentation techniques are employed to detect some predefined objects in the image. An approach proposed by Chang et al. [11] uses the Semantic Visual Templates (SVTs), a collection of regional objects within a video shot, to express the semantical concept of a user's query. The templates can be further refined by a two-way interaction between the user and the system. Saber and Tekalp [40] propose extracting objects from an image based on color and edge detection. Recently, Wang et al. [50] propose SIMPLIcity, a system that captures semantics using the robust Integrated Region Matching metric. The semantics are used to classify images into broad semantical categories "textured/nontextured" and "graph/photograph". The classification is then used to support semantics-sensitive image

retrievals. IBM Research [34] develops VideoAnn, a semi-automatic video annotation tool. The tool provides an easy-to-use interface for the user to annotate different regions of a video shot. The annotation is then automatically propagated to similar shots.

2. Text-based Methods. The text surrounding the images is analyzed (e.g. [42]) and the system extracts those that seem to be relevant. Benitez and Chang [5] present a method to extract semantical concepts by disambiguating words' senses with the help of the lexical database WordNet. In addition, the relationships between keywords can be extracted using relations established in WordNet. An approach proposed by Chang et al. [36] combines text-based and image-based methods to attach annotations to images. Their system is able to separate images into two main semantical concepts, indoor and outdoor.

After the semantics are extracted, Naphade and Huang [33] propose a probabilistic framework to represent them in their video indexing and retrieval work. A multiject is the probability density function which denotes the presence of a multimedia object. The interaction between the multijects is then modeled using a Bayesian network to form a multinet.

All the approaches mentioned above rely very much on local features, which in turn rely on high segmentation accuracy. However, segmentation can hardly be done reliably, especially on compressed images. CBSA performs annotation using global features, and it uses a BPM-OPC ensemble to provide multiple semantical labels for an image without the need of segmentation. Our empirical results show that using global features, we can perform salient object recognition and semantical understanding reasonably well. On a 116-category 25K-image dataset, using 20%, 30%, and 50% images as training data, the annotation accuracy on the remaining testing data is 49%, 56%, and 61%, respectively.

2.2.2 Annotation Propagation

Most image annotation techniques involve some sort of propagation. A small subset of images is first manually annotated. Then the annotation is propagated to the rest of the database using some machine learning or statistical methods. Picard and Minka [37] use image texture to propagate annotation. After a user labels a patch of an image, this label can be propagated to other images with similar patches of texture. In [40], after an object is extracted from an image, it is compared to a set of predefined object templates and, if a match is found, the template's annotation is used for that object. Naphade, Lin and Smith [35] present a system which contains a lexicon learnt from users using annotated examples. The formulation of semantical relationships in the lexicon is modeled as a classification problem. The resulting retrieval system is able to support keyword-based queries as well as to detect multimodal events.

Wenyin et al. [52], Sychay et al. [44] and VideoAnn [34] use relevance feedback to improve

annotation quality. After a user types in keywords for a query, the user picks some positive and negative examples. Positive examples have the query keywords added or strengthened, and negative examples have the keywords weakened or removed. Such schemes depend on the availability of a reasonable set of initial image labels, which CBSA can provide both effectively and efficiently.

3 The Choice of A Classification Scheme

The regression approach to classification considers the discrimination problem of K classes simultaneously. However, as discussed in Section 2, the regression approach is computationally expensive ($O(K^3)$) and may impose a universal assumption on all class boundaries. A more flexible and computationally inexpensive ($O(K^2)$) approach is the use of ensemble schemes. Such schemes first decompose the K-class problems into a number of binary sub-problems and use a simpler binary classifier to solve the smaller sub-problems. For conducting content-based soft annotation, we use Bayes Point Machines (BPMs) as the binary classifer and a simple yet effective ensemble scheme known as *one per class*. We call this combination BPM-OPC.

In this section, we first present two statistical learning methods, Support Vector Machines (SVMs) and BPMs. Since SVMs can be regarded as an approximation to BPMs and can be used to bootstrap BPMs, we present SVMs first to set up the context for discussing the theoretical differences between these two emerging learning methods. Next, we present a kernel parameter selection algorithm. Both SVMs and BPMs can employ a kernel function to project training data to a higher-dimensional feature space where the data from different classes can be separated by a hyperplane. We discuss the proper kernel used for classifying images, and present an algorithm to adaptively tune the kernel parameters so as to minimize generalization error. Finally, we present details of the OPC ensemble scheme.

3.1 Learning Methods: SVMs and BPMs

SVMs [23, 48, 49] are a core machine learning technology that has strong theoretical foundations and excellent empirical successes. We consider SVMs (and thus BPMs) in the binary classification setting. We are given training data $\{\mathbf{x}_1 \ldots \mathbf{x}_n\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^d$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin (see Figure 1). All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space $\mathcal{X}$ to a higher dimensional feature

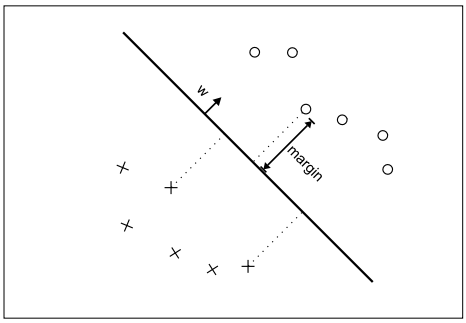


Figure 1: A Simple Linear Support Vector Machine.

space $\mathcal{F}$ via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \mathbf{x})$. When $f(\mathbf{x}) \geq 0$ we classify $\mathbf{x}$ as $+1$; otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [8] we can write: $K(\mathbf{u}, \mathbf{v}) = \Phi(\mathbf{u}) \cdot \Phi(\mathbf{v})$ where $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \Phi(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{i=1}^{n} \alpha_i \Phi(\mathbf{x}_i). \tag{1}$$

The SVM computes the α_is that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions we can project the training data from $\mathcal{X}$ into different spaces $\mathcal{F}$. The hyperplanes in $\mathcal{F}$ correspond to complex decision boundaries in the original space $\mathcal{X}$. Commonly used kernel functions include polynomial kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + 1)^p$, Gaussian radial basis function (RBF) kernel $K(\mathbf{u}, \mathbf{v}) = (e^{-\gamma(\mathbf{u}-\mathbf{v})\cdot(\mathbf{u}-\mathbf{v})})$ and Laplacian RBF kernel $K(\mathbf{u}, \mathbf{v}) = (e^{-\gamma \sum_i |u_i - v_i|})$.

Since image cencepts are not likely to be linearly separable in the projected space, we need to allow for some training errors. This need gives rise to the *soft margin* SVM algorithm [13]. Soft margin SVMs can be formulated as a special case of the hard margin version with the modified kernel by adding a factor C to penalize training errors.

Bayes Point Machines (BPMs) are proposed by Herbrich et al. [19, 20] to approximate the Bayesian inference for linear classifiers in a kernel space. To illustrate the difference between BPMs and SVMs, we first define some terms. A *hypothesis* $h(x)$, or commonly referred to as a classifier, is a function which maps an input vector x to an output label y. The set of consistent hypotheses given a training set is called the *version space* $\mathcal{V}$ and each point in the version space constitutes a plausible classifier. An SVM hypothesis is constructed by finding a hyperplane which maximizes the margin between the positive and negative training examples. In the version space, the margin of examples is the distance from the classifier to the closest boundary of the

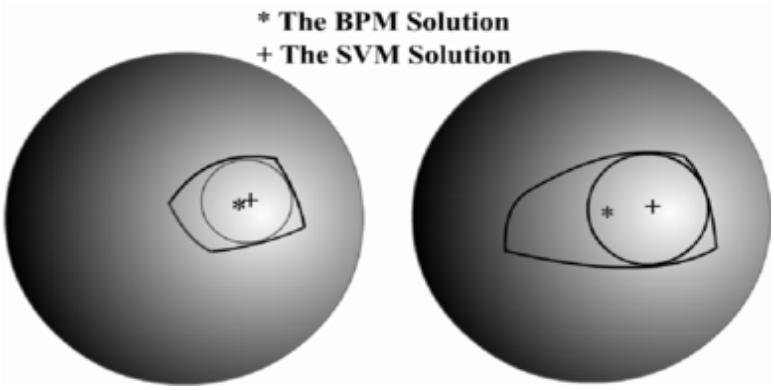


Figure 2: Difference between SVM and BPM classifiers. For the hypersphere on the left, SVM approximates the Bayes point (star symbol) with the center of the largest sphere that is embedded within the version space. However, for the hypersphere on the right, we see that the SVM solution is far from the Bayes point. This happens when the largest embedded sphere does not cover a large volume in the version space.

space. Thus, the maximal margin is the radius of the largest sphere that can be embedded in the version space, and the SVM classifier is the center of this sphere. In contrast, BPMs construct a hypothesis by attempting to find the center of the entire version space. This center is called the *Bayes point*. Figure 2 illustrates a scenario whereby the hypothesis obtained by SVMs can be quite inaccurate with respect to the theoretically optimal Bayes point. Next, we discuss how the Bayes point is computed.

3.1.1 The Bayes Solution and Bayes Point

BPMs adopt the strategy of Bayes classification, where the classifier is found by considering all valid solutions over the entire version space. Given a training set $\mathbf{z} = (\mathbf{x}, \mathbf{y}) = ((x_1, y_1), \ldots (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ of size m, Bayes classification aims to assign a new test data x to the label y with minimal expected loss, weighted by its posterior probability $P_{H|Z^m=z}(h)$:

$$Bayes_z(x) = \arg\min_{y \in \mathcal{Y}} E_{H|Z^m=z}[l(H(x), y)], \tag{2}$$

where the loss function is defined by

$$l(y, y') = \begin{cases} 1, & y \neq y' \\ 0, & y = y' \end{cases}$$

The Bayes solution ($Bayes_z$) in Equation 2 is the optimal one for any classification task. However, it is generally impossible to find a unique hypothesis $h \in \mathcal{H}$ in order to obtain $Bayes_z$. Herbrich et al. [19, 20] attempts to find the classifier that is the closest to $Bayes_z$ by requiring the classifier

to lie within a fixed hypothesis space H. The $Bayes_z$ approximation is given by

$$h_{bp}(z) = \arg\min_{h \in \mathcal{H}} E_X \lfloor E_{H|Z^m=z}[l(h(X), H(X))] \rfloor = \langle w_{Bayes}, X \rangle_F. \tag{3}$$

The classifier $h_{bp}(z)$ is called the *Bayes point*, and it approximates the optimal $Bayes_z$ the best. Here, w_{Bayes} is the weight vector of $h_{bp}(z)$.

3.1.2 The Center-of-Mass

Computing Equation 3 requires the knowledge of the input training distribution P_x, which is hard to obtain. Fortunately, for a two-class classification problem, [20] suggests a method to find the center-of-mass in $\mathcal{V}$, under the assumption that $\|w\| = 1$, as follows

$$w_{cm} = \frac{E_{W|Z^m=z}[W]}{\|E_{W|Z^m=z}[W]\|}. \tag{4}$$

Note that the hypothesis $h \in \mathcal{H}$ and the weight vector $w \in \mathcal{W}$ can be synonymously called the classifier as they have a one-to-one correspondence [19, 20]. Equation 4 can be a very good approximation to the Bayes point $h_{bp}(z)$, and thus, we can find the optimal weight vector w_{Bayes} by calculating w_{cm}. In order to calculate the center-of-mass w_{cm}, [20] suggests using a first-order Markov chain to compute the expectation in Equation 4, which can be approximated by uniformly sampling weight vectors w in $\mathcal{V}$ and then averaging them. However, it is rather difficult to perform such samplings as the points exist in a convex polyhedron on a unit sphere. Two algorithms proposed by [20] to achieve this sampling are described as follows:

1. The Billiard-ball algorithm. The algorithm is based on a billiard-like method [38, 39]. After entering the version space with a perceptron or the SVM algorithm, the classifier w is envisioned as a billiard ball that is being bounced around within the convex polyhedron $\mathcal{V}$ for a while. Playing billiards in $\mathcal{V}$ is possible because each training point $(x_i, y_i) \in \mathcal{Z}$ defines a hyperplane $\{w \in \mathcal{W} | y_i \langle x_i, w \rangle_F = 0\} \in \mathcal{W}$ (see Figure 3). After N bounces of the billiard ball, the Bayes point can be estimated by $w_{cm} = \frac{1}{N} \sum_{i=1}^{N} w_i$. More details can be gleaned from [20].

2. The perceptron algorithm. While the billiard-like algorithm shows good generalization performance, it suffers from large computational costs. In [20], it is suggested that an approximate uniform sampling of the version space is sufficient for finding w_{cm}. This suggestion is based on the fact that each classifier in $\mathcal{V}$ is already optimal. Hence, the second method uses the perceptron algorithm to obtain different classifiers w_is simply by permutating the training set [51]. The center of mass is given by averaging over all w_is.

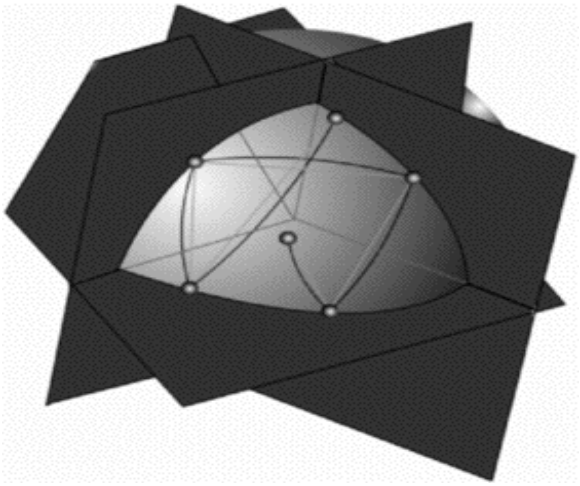


Figure 3: Illustration of the billiard-like algorithm. Each small ball represents the estimate w_i for the center of mass w_{cm} in each bounce step. The final w_{cm} is obtained by averaging all these estimates.

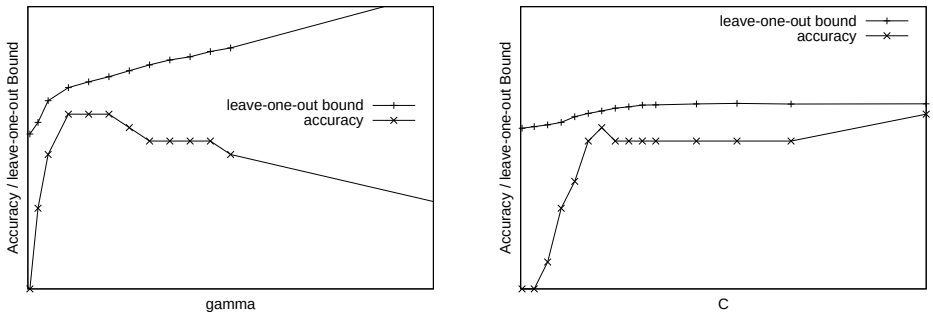
For our BPM-OPC implementation, we use SVMs to delimit the version space and then use the Billiard-ball algorithm to approximate the center-of-mass.

3.2 Kernel and Parameters Selection

For image classification, recent works [1, 21] show that using the L_1 distance function is a more effective similarity measure for high-dimensional data. Meanwhile, the empirical result in [46] shows that the Laplacian RBF, which utilizes the L_1 distance function, works well with high-dimensional image data. Therefore, CBSA uses Laplacian RBF kernel. Because of the choice of using the Laplacian kernel together with soft-margin SVMs to initialize BPMs in the version space, our ensemble has two tunable parameters: γ and C.

- The γ in the Laplacian kernel controls the shape of the kernel. The larger the value of γ, the narrower the shape of $K(\mathbf{u}, \mathbf{v}) = (e^{-\gamma \sum_i |u_i - v_i|})$. A high γ value leads to a narrow shape and possible overfitting. A lower value (wider shape) produces low prediction accuracy.
- C controls the trade-offs between margin maximization and error minimization. Increasing C may decrease training error, but it can also lead to poor generalization.

Both parameters control the generalization ability of BPMs and SVMs. To algorithmically predict the best set of the parameters that will minimize the generalization error is an open research problem. The machine learning community has provided some bounds, produced from



(a) γ (b) C

Figure 4: The *leave-one-out* bound indicates good parameters.

training results, to estimate the generalization error. One widely accepted approach at present is the *leave-one-out* bound [28]. The leave-one-out procedure removes one instance from the training set to be used as the testing instance. Suppose a training set consists of L training instances. The procedure first constructs the decision rule based on the remaining $(L-1)$ training data, then tests the rule with the testing instance. This procedure is repeated L times until all L instances in the training set are tested. The leave-one-out error is the average error of the L tests. It is known that the leave-one-out procedure gives an almost unbiased estimate of the expected generalization error. Unfortunately, it is computationally intensive to run the training L times.

The study of [12] summarizes a few practical approximations of the leave-one-out error. Among the choices, we use

$$T = N_{SV}/L,$$

where N_{SV} denotes the number of support vectors, to estimate the generalization error. Although Burges [8] shows that the parameter setting that maximizes T does not necessarily minimize generalization error, we find that the first derivative of T (denoted as T') with respect to γ and C useful in the following ways:

- $T'(\gamma) = \partial T/\partial\gamma$. When we incrementally increase γ for an RBF kernel, T increases. Near the γ where $T'(\gamma)$ encounters a sharp decline, the γ setting tends to produce minimal generalization error (or maximal accuracy as shown in Figure 4(a)). We believe this is because the problem of overfitting may start taking place after that point.
- $T'(C) = \partial T/\partial C$. When the increase of C does not lead to an increase in the number of support vectors, further increasing C is counter-productive for two reasons. First, it may not further improve generalization error (as shown in Figure 4(b)). Second, the larger the

C, the longer the training time. Therefore, it is most cost effective to choose the C where $T'(C)$ flats out.

Based on the above observations, we propose the algorithm depicted in Figure 5 to adaptively select optimal kernel parameters. The algorithm begins with default parameters, then repeatedly trains with the training images in order to find a good γ setting. Once a good γ setting is found, the algorithm can find a good C setting.

3.3 One-Per-Class Ensemble Scheme

A classification problem involves a function $F : \Omega \rightarrow 1, \ldots, K$, which defines a K-partition of the input space Ω into sets $F^{-1}(k)$ called *classes* denoted by ω_k, where $1 \leq k \leq K$. For a two-class problem, the hyperplane $\mathbf{w}$ of SVMs (and hence BPMs) is the partition and $sgn(f(\mathbf{x}))$ is equivalent to $F^{-1}(k)$.

Although SVMs/BPMs are mainly applied to two-class problems, they can be adapted to multi-class problems by ensemble schemes. One such scheme is called *one per class* (OPC) or "one against others". OPC is simple and gives satisfactory performance. When the number of classes K is large, we need only to train K binary classifiers for OPC. The PWC scheme requires $\frac{1}{2}K(K-1)$ binary classifiers. And for ECOC schemes to have good error-correction capability, the length of the codewords (which determines the number of binary classifiers) needs to be much larger than K. Requiring a large number of classifiers not only impacts the training time, but also the speed of classification. OPC is the fastest ensemble scheme, and yet its classification performance compares favorably to that of the more complex schemes.

For K classes, we train K classifiers, each of which separates one class from the other $(K-1)$ classes. For each point x, we have K decision outputs $f_k(\mathbf{x})$, $1 \leq k \leq K$. The class of a point is given as $\arg\max_k f_k(\mathbf{x})$.

4 Image Annotation

In this section, we first explain how the CBSA method assigns labels to unannotated images using BPM-OPC. We then describe the image features used for propagating image labels.

4.1 Content-based Soft Annotation (CBSA)

Image classification systems work by classifying an image into one of many predefined categories. Usually, a confidence score is associated with each category. The highest one is chosen as the

ALGORITHM Select_SVM_Parameters

input
D: training set of images
ϵ, θ, η
output
γ_{opt}: optimal RBF width
C_{opt}: optimal training error penalty
variables
γ_i: RBF width for iteration i
C_i: training error penalty for iteration i
T_i: leave-one-out bound ($\frac{N_{SV}}{L}$) for iteration i
OPC_i: SVM classifier for iteration i
procedure calls
TRAIN_SVM
CALCULATE_T

init
// initialize γ and C with small values
$\gamma_0 \leftarrow \gamma_{default}$
$C_0 \leftarrow C_{default}$
$T_0 \leftarrow 0$
$i \leftarrow 0$
end init

repeat
// increase γ
$i \leftarrow i + 1$
$\gamma_i \leftarrow \gamma_{i-1} \times \epsilon$
$OPC_i \leftarrow$ TRAIN_SVM (D, γ_i, C_0)
$T_i \leftarrow$ CALCULATE_T (f_i)
until $T_i - T_{i-1}/T_{i-1} \leq \theta$
$\gamma_{opt} \leftarrow \gamma_i$

repeat
// increase C until T flattens out and training error is 0%
$i \leftarrow i + 1$
$C_i \leftarrow C_{i-1} \times \eta$
$OPC_i \leftarrow$ TRAIN_SVM (D, γ_{opt}, C_i)
$T_i \leftarrow$ CALCULATE_T (f_i)
until $T_i - T_{i-1} \leq 0$ **AND** training error is 0%
$C_{opt} \leftarrow C_i$

return γ_{opt} and C_{opt}

Figure 5: Algorithm for Selecting SVM Kernel Parameters.

image's category, and the rest are ignored. However, in CBSA, we are interested in *all* category scores.

We define a set of 116 labels where each label characterizes the representative semantics of an image category. Table 5 in Appendix A shows the category names and the number of images belonging to each. The eight top-level categories for all of the images are: *abstract*, *architecture*,

animal, *food*, *landscape*, *objects*, *people* and *plants*. Each unannotated image is classified against the 116 categories using BPM-OPC. This produces a ranking of the 116 categories, with each category assigned a confidence probability. The labels of these 116 categories, along with their probabilities, become the annotation for this image. The probability represents the weight of a label in the overall description of an image. The rationale is that if an image of, say, a landscape with clouds in the sky is being classified, the classifier will assign the highest probability to the landscape label and the second highest probability to the clouds label. This ranking can be very useful in retrievals. After an image is annotated, it is associated with a K-nary label-vector. Each element in the vector is a keyword, and the value for that element is the weight of the keyword. A typical vector may look like

$$\{(landscape, 0.5), (cloud, 0.7), ..., (tiger, 0.9)\}.$$

4.2 Image Features

This section describes how our system characterizes images. These image features help propagate annotation from trained images to unlabeled images.

We believe that image characterization should emulate human perception insofar as possible [18]. Therefore, our perception works in a multi-resolution fashion. For some visual tasks, human eyes may select coarse filters to obtain coarse image features; for others, they select finer features. Similarly, for some image applications (e.g., for detecting image replicas), employing coarse features is sufficient; for other applications (e.g., for classifying images), employing finer features may be essential. An image search engine thus must have the flexibility to model subjective perceptions and to fulfill a variety of search tasks.

In our image retrieval system [10], we characterize images by two main features: color and texture. We consider shape as an attribute of these main features.

4.2.1 Color

Although the wavelength of visible light ranges from 400 nanometers to 700 nanometers, research [18] shows that the colors which can be named by all cultures are generally limited to eleven. In addition to *black* and *white*, the discernible colors are *red*, *yellow*, *green*, *blue*, *brown*, *purple*, *pink*, *orange* and *gray*.

We first divide color into 12 color bins including 11 bins for culture colors (those common to all cultures) and one bin for outliers [22]. At the coarsest resolution, we characterize color using a color mask of 12 bits. To record color information at finer resolutions, we record eight additional features for each color. These eight features are color histograms, color means (in

H, S and V channels), color variances (in H, S and V channel), and two shape characteristics: elongation and spreadness. Color elongation characterizes the shape of a color, and spreadness characterizes how that color scatters within the image [24]. Table 1 summarizes color features in coarse, medium and fine resolutions.

Filter Name	Resolution	Representation
Masks	Coarse	Appearance of culture colors
Spread	Coarse	Spatial concentration of a color
Elongation	Coarse	Shape of a color
Histograms	Medium	Distribution of colors
Average	Medium	Similarity comparison within the same culture color
Variance	Fine	Similarity comparison within the same culture color

Table 1: Multi-resolution Color Features.

4.2.2 Texture

Texture is an important cue for image analysis. Studies [30, 43, 45, 29] have shown that characterizing texture features in terms of structuredness, orientation, and scale (coarseness) fits well with models of human perception. A wide variety of texture analysis methods have been proposed in the past. Because of its computational efficiency, we choose a discrete wavelet transformation (DWT) using quadrature mirror filters [43].

Each wavelet decomposition on a 2-D image yields four subimages: a $\frac{1}{2} \times \frac{1}{2}$ scaled-down image of the input image and its wavelets in three orientations: horizontal, vertical and diagonal. Decomposing the scaled-down image further, we obtain the tree-structured or wavelet packet decomposition. The wavelet image decomposition provides a representation that is easy to interpret. Every subimage contains information of a specific scale and orientation and also retains spatial information. We obtain nine texture combinations from subimages of three scales and three orientations. Since each subimage retains the spatial information of texture, we also compute elongation and spreadness for each texture channel. Figure 6 summarizes texture features.

Now, given an image, we can extract the above color and texture information to produce a 144-dimension vector of numbers. We use this vector to represent the image. Thus, the input space $\mathcal{X}$ for our BPM classifiers is a 144-dimension space, and each image in our database corresponds to a point in this space.

5 Experimental Results

Our experiment is divided into two major parts:

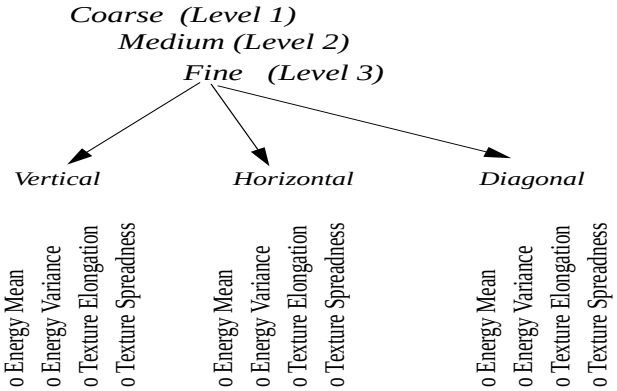


Figure 6: Multi-resolution Texture Features.

1. **Classification Schemes Evaluation**. We evaluate the performance of our adaptive parameters selection algorithm. Then using the best parameters setting, we compare the classification performance of SVM-OPC with that of BPM-OPC's.

2. **Soft Annotation**. We first assign a bag of probabilities derived from BPM-OPC to each test image. We then test the correctness of this soft annotation by eyeballing each test image.

For our empirical studies, we use a 51,000-image dataset that contains images accumulated from the Corel Image CDs and the Internet. Corel images have been widely used by the computer vision and image-processing research communities. In order to fulfill our experimental goals, we further divide the dataset into the following two subsets:

- **2K-image dataset**. This smaller set of images is used to evaluate the BPM and SVM classifiers using various kernel parameters. This set contains representative images from fifteen Corel categories — *architecture*, *bears*, *clouds*, *elephants*, *fabrics*, *fireworks*, *flowers*, *food*, *landscape*, *people*, *objectionable images*, *textures*, *tigers*, *tools*, and *waves*. A few categories were added to increase learning difficulty for the classifiers. To make the landscape category more difficult to learn, we placed images of tigers and elephants on landscape and water backgrounds. The "objectionable image" category had to be differentiated from the "people" category when an image was of people wearing little clothing. We added colorful fabrics and food to interfere with flowers. Various texture images (e.g., skin, brick, grass, water, etc.) were added to raise the learning difficulty for all categories.

- **25K-image dataset**. This set contains images that were manually classified into 116 categories. (See Table 5 in Appendix A for details.) This set contains images that are more diversified and thus more challenging for any learning algorithm. Using this dataset, we

show that our classification scheme is scalable to a large number of categories. We also use this dataset to train the classifiers that provide probabilities used in soft annotations.

5.1 Classification Schemes Experiments

The classification experiment is divided into three parts:

1. We first conduct an extensive analysis of the parameters γ and C using SVMs with the Laplacian RBF kernel. The adaptive parameters selection algorithm is used to find a good set of parameters for the second part of the experiment.
2. Next, we compare the classification performance of SVM classifiers with that of BPM's. The first two parts of the experiment make use of the 2K-image dataset.
3. Finally, we evaluate the scalability and performance of both SVM and BPM classifiers on the 25K-image dataset.

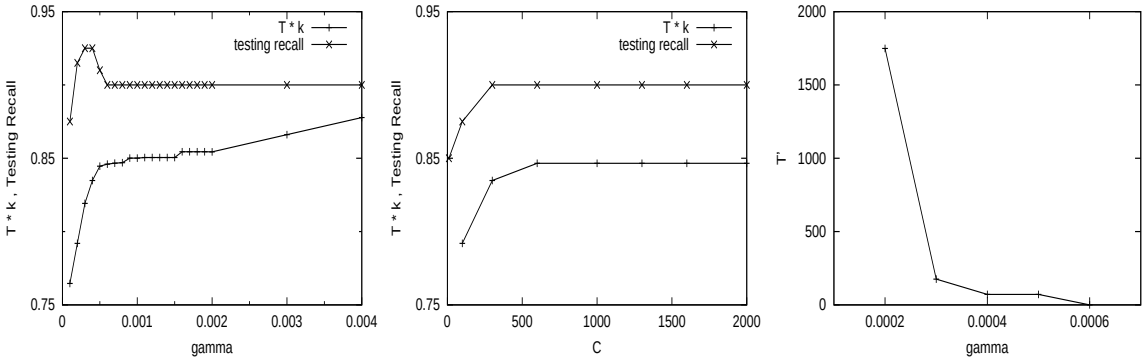
To measure classification performance in our experiments, we used classification error. We ran each experiment 10 times. At each run, we randomly selected a percentage of images from the dataset to be used as training examples for the classifiers. The remaining images were used for classification. The results presented represent the average of 10 runs.

5.2 γ and C Analysis

For analytical purposes, we look at how γ and C affect classification results in a number of ways. For various combinations of γ and C, we train SVM classifiers on 80% of the data and test on the remaining 20%. We do this for each of the 15 classes. For each class, we gather the training and testing recall, and the *leave-one-out* bound T. Recall is defined as n/p where n is the number of positive instances correctly labeled (i.e., annotated) positive, and p is the number of positive instances in the dataset. We examine the effectiveness of the adaptive algorithm by evaluating whether the selected kernel parameters can improve SVM-OPC's testing recall.

In Figure 7, we plot $T \times k$ (where k is a scaling factor) along with testing recall for a representative class. We vary γ and fix C at 1000. Figure 7(a) shows that the leave-one-out bound $T \times k$ always increases when γ is increased, but the "knee" in the curve predicts the optimal value of $\gamma \approx 0.0006$ for this class. (Recall that $\gamma = 1/\sigma$ so in Figure 7(a), we are decreasing the RBF width as N_{SV} grows.)

Figure 7(b) shows a similar experiment, but we fix γ at 0.0006 and vary C. We graph both testing recall and $T \times k$. As C increases, $T \times k$ increases and then flattens out. The value for C



(a) The shape of T predicts the optimal γ.

(b) The shape of T indicates a good value for C.

(c) The algorithm increases γ until T' is very small.

Figure 7: Data for a representative class. T is scaled up by a factor $k = 6$.

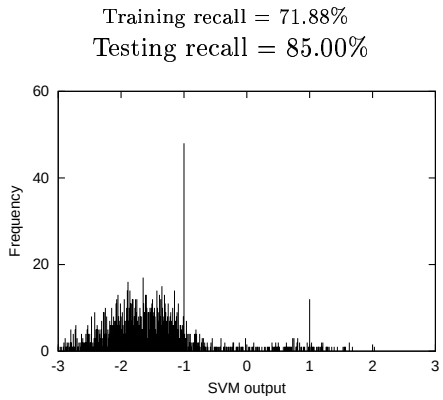
where the bound flattens out closely matches the value for C that optimizes testing recall. When C increases further, the recall remains unchanged.

In Figure 7(c), we plot T' as we increase γ. The γ values that correspond to a very small T' correctly approximate the optimal $\gamma \approx 0.0006$ from Figure 7(a). Thus, our algorithm picks a good value for γ.

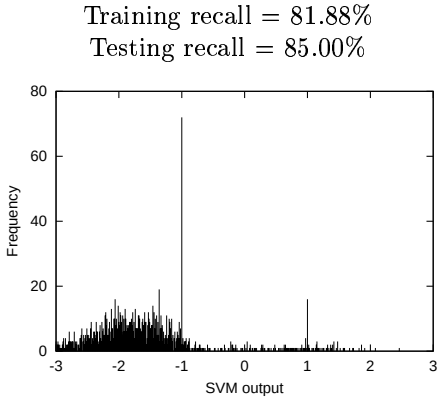
Figures 8(a-e) illustrate some iterations in the algorithm. In each figure, we present the distribution of the SVM training output, and we also display the training recall and testing recall. Figure 8(a) shows that when γ is sub-optimal, the trained classifier cannot cleanly separate the training data into their classes: several training instances fall within the SVM margin (between -1 and 1 in the x-axis). When we increase γ (Figures 8(b) and (c)), the number of instances in the margin decreases, and so does the training error. We fix $\gamma = 0.0006$ when T reaches its knee. We then increase C from 100 to 300 (Figure 8(d)), and then to 400 (Figure 8(e)). At $\gamma = 0.0006$ and $C = 400$, the testing recall achievs the best result (Figure 8(f)). Using the algorithm from Figure 5, we select a γ and C for each of 15 classes (see Table 2).

5.3 Classification Performance of SVM-OPC vs. BPM-OPC

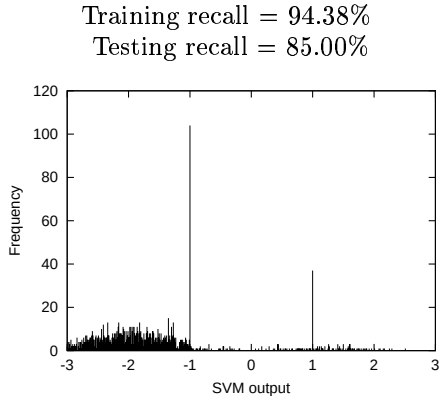
Using the γ and C parameters from Table 2, we evaluate the classification performance of SVM-OPC and BPM-OPC classifiers. We use 80% of the 2K-image dataset for training the classifiers, and the remaining 20% are used for testing. For BPM-OPC, the tolerance parameter for termination of the billiard algorithm is set at 0.01. Table 3 shows the classification error rate for each of the 15 categories. While the error rates for SVM-OPC are already low, we observe that



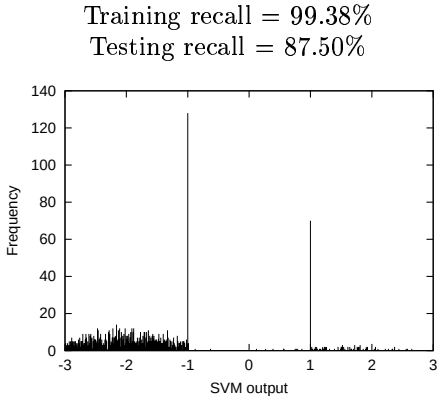
(a) $\gamma = 0.0001$ $C = 100$



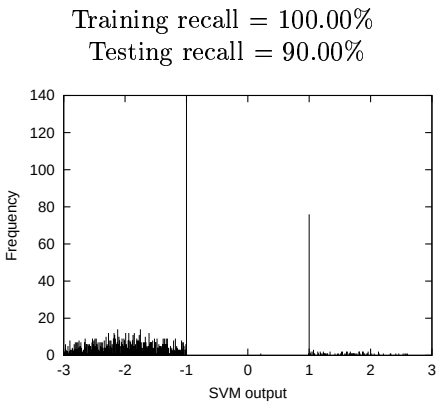
(b) $\gamma = 0.0002$ $C = 100$



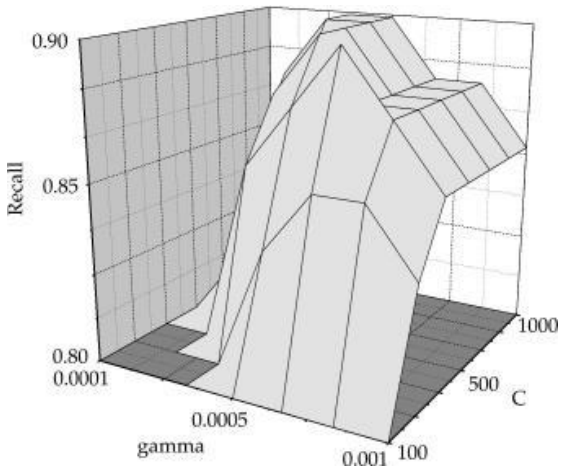
(c) $\gamma = 0.0006$ $C = 100$



(d) $\gamma = 0.0006$ $C = 300$



(e) $\gamma = 0.0006$ $C = 400$



(f) Search space for the best γ and C

Figure 8: Each iteration improved classification.

Category	γ	C
architecture	0.0013	300
bears	0.0004	300
clouds	0.0013	200
elephants	0.0003	400
fabric	0.0003	300
fireworks	0.001	200
flowers	0.0006	400
food	0.0012	300
landscape	0.0004	300
obj. images	0.0021	300
people	0.0003	300
texture	0.0011	200
tigers	0.0012	300
tools	0.0007	200
waves	0.0024	300

Table 2: Adaptively generated parameters for each class.

BPM-OPC can obtain an even lower error rate for all categories. The decrease is about 1.6% on average. However, the training time for a BPM classifier is substantially longer than that of an SVM classifier.

Next, we apply our classification schemes on the larger 116-category dataset. We set aside $4K$ images to be used as testing data; the remaining $21K$ images are used for training. We perform several rounds of SVM-OPC and BPM-OPC classification, with each round using a different percentage of training images. The generated classifiers are tested on the 4K-image testing set.

Figure 9 shows the classification error of SVM- and BPM-OPC for the test set. Despite having 116 categories, many of them closely-related, both SVM and BPM classifiers are still able to provide acceptable classification performance. Even with just 20% of the training data, we maintain a relatively low classification error rate of only 60% (SVM-OPC) and 51% (BPM-OPC). As we increase the training percentage to 50%, the errors drop to 47% for SVM-OPC and 39% for BPM-OPC. The performance gap between BPM-OPC and SVM-OPC of the 116-category dataset is significant (about 10%). Although training BPM-OPC takes four-times longer than training SVM-OPC, BMP-OPC is a better choice for CBSA, which can conduct class prediction offline.

Next, we analyze the effect of category size on error rates. We divide the categories into three groups: the small group's categories has fewer than 50 images, the medium group's categories has fewer than 300 images, and the large group has categories with more than 300 images. Table 4 shows the average error rates of the categories in the three groups when 50% of the dataset is used for training. We see that the error rate for the small group is much worse (30% for SVM-OPC and 21% for BPM-OPC) than for the large group, while the difference between the

Category	SVM-OPC	BPM-OPC
architecture	12.0	11.3
bears	25.3	25.1
clouds	15.4	12.3
elephants	16.1	14.7
fabric	14.9	9.9
fireworks	6.5	4.9
flowers	8.7	6.8
food	7.8	6.5
landscape	21.2	19.8
obj. images	4.9	4.1
people	17.7	15.9
texture	3.5	3.4
tigers	8.5	6.8
tools	0.5	0.5
waves	13.9	10.4
Overall Average	**11.8**	**10.2**

Table 3: Classification Error Rates for SVM-OPC and BPM-OPC using 80% of the 2K-image Dataset for Training.

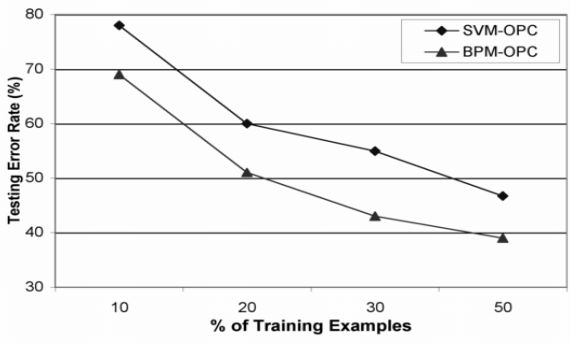


Figure 9: Error Rates of the 4K-image Testing Set.

medium and large groups is, although smaller, still rather compelling. This phenomenon is due to class imbalance whereby the large group can provide more positive examples, and hence are well represented in the training set. For the small group, the number of positive examples in the training set is insignificant compared to the number of negative examples, and thus the class boundary formed by the classifier may not generalize well.

5.4 Soft Annotation Results

Using the classifiers generated with 20% of training data from the 25K-image dataset, we produce a set of BPM-OPC values for each unannotated image. The values are converted to probabilities in order to depict the likelihood of a category label describing an image correctly. A probability

Percentage of Training	SVM-OPC	BPM-OPC
Small (< 50 images)	73.6	58.0
Medium (< 300 images)	53.1	41.5
Large (≥ 300 images)	43.6	36.9

Table 4: Error Rates for Different Category Sizes with 50% of Dataset used for Training.

greater than 0.9 implies that the annotation shows a high level of confidence, whereas a probability less than 0.1 implies a low confidence level. Probabilities that are greater than 0.5 may also provide some useful secondary annotation of an image. The first label is the one which gives the highest probability (denoted as P_1) and the second label is the one with the second highest probability (P_2).

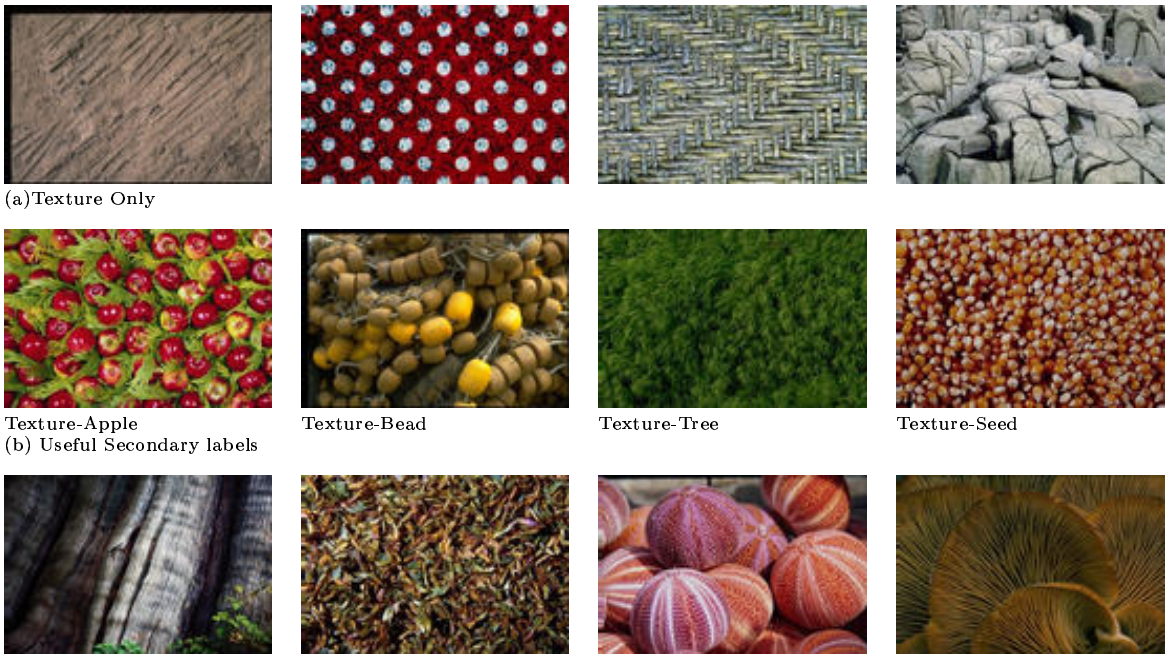


Figure 10: Examples of Texture Query. (a) Images that are confidently annotated. (b) Images with useful secondary labels. (c) Images with wrong secondary labels.

We present three examples of the annotation results. The first example is for *texture* images (Figure 10), the second is for *mountain* images (Figure 11) and the last example is for *city* images (Figure 12). Each row of images in the figures portrays one of the following scenarios:

Case 1: $P_1 \geq 0.9$. (See row(a) of Figures 10, 11, and 12.)
The first label is able to provide annotation of which we may be confident. We can see from all the examples in row(a) that the label accurately describes the images. The second label is not

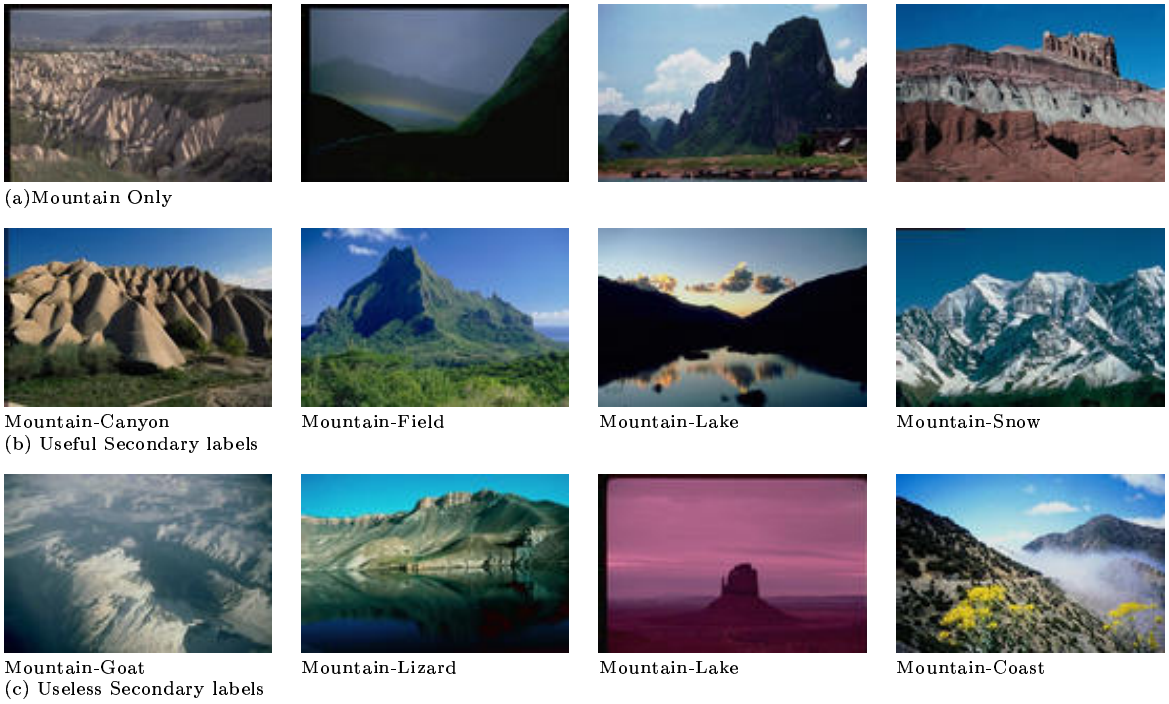


Figure 11: Examples of Mountain Query. (a) Images that are confidently annotated. (b) Images with useful secondary labels. (c) Images with wrong secondary labels.

used at all because the first label supersedes all other labels.

Case 2: $P_1 < 0.9$ and $P_2 > 0.5$. (See row(b) of Figures 10, 11, and 12.)
In this case, P_1 is high but P_2 is greater than 0.5. We see from images in row(b) that the second label can enrich the description of the image. For the *texture* images (Figure 10(b)), the second label adds a semantical context to the otherwise abstract *texture* label. Some secondary annotations (such as *snow*, which is semantically close to the word *mountain*), frequently give additional information. However, a more general label like *lake* cannot be trusted all the time.

Case 3: $P_1 < 0.9$ and $P_2 < 0.5$. (See row(c) of Figures 10, 11, and 12.)
In this case, we see examples where the first label correctly annotates an image, but the secondary annotation seems completely wrong. The probability P_2 in this case is usually less than 0.5.

Remarks

- Using just 20% of labeled data to train BPM-OPC classifiers, we can achieve label propagation to un-annotated images with a reasonable accuracy of 50%. Thus, CBSA provides a suitable initial labeling that can further be improved by using methods such as users' relevance feedback.

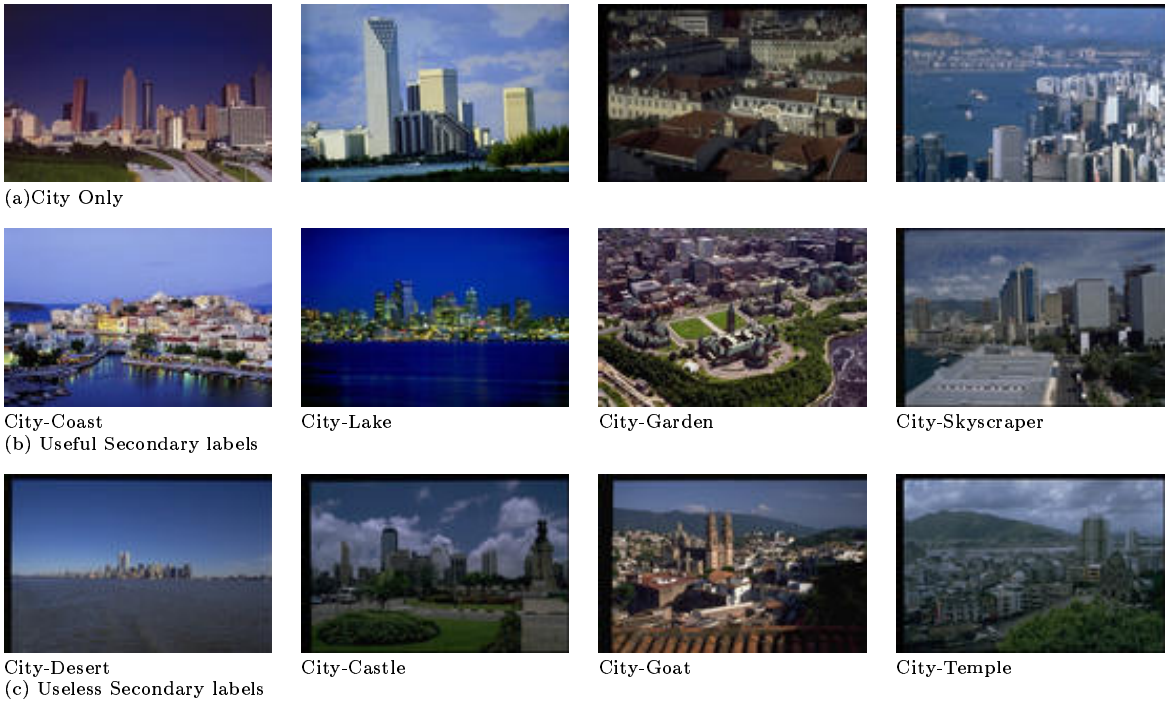


Figure 12: Examples of City Query. (a) Images that are confidently annotated. (b) Images with useful secondary labels. (c) Images with wrong secondary labels.

- Multiple-keyword searches can be supported when the combined confidence level of both the first and second label is high. The combined confidence level is computed by multiplying P_1 with P_2. We obtain a high combined confidence level when both P_1 and P_2 are greater than 0.5, and are close to each other. If $(P_1 \geq 0.9)$ and $(P_2 < 0.5)$, the high confidence level of P_1 is diluted by P_2 in the combined probability. Thus, we do not make use of the second label. In addition to the diluting effect, $(P_2 < 0.5)$ indicates a low confidence in annotation by the second label. When the value P_2 is low, we discard the second label in our annotation.

6 Conclusion and Future Work

In order to support multimodal image retrievals, we have presented CBSA to semi-automate the process of annotating unlabeled images with multiple soft labels. CBSA first uses a small percentage of manually-labeled data to train a BPM-OPC ensemble. The ensemble is then applied to all images and the ensemble's outputs mapped to probabilities. Every image is characterized by a bag of words. The words' relevancy to the image is indicated by probability values. Our approach of associating probabilities with the labels establishes a baseline whereby the initial

labeling can be improved upon through users' relevance feedback via active learning [9, 44, 46].

Our future work can be expanded in two directions. First, we will improve the query effectiveness of our system by adding query expansion using a text lexical system (e.g., WordNet or a domain-specific thesaurus). In this way, a query will not be confined to the initial labels. Second, we will improve the effectiveness and efficiency of the BPM-OPC ensemble. For effectiveness enhancement, we are investigating statistical methods that can deal with the challenges of an unbalanced training set (where one class is substantially smaller than the other in a binary classification setting). For efficiency enhancement, we are investigating methods for improving computational time to approximate the Bayes point. Together with our recent works in perceptual feature mining [27], perceptual distance functions [25, 26], active learning [9, 46], and indexing methods [17], we believe that CBSA provides the glue to integrate content with keywords for supporting effective multimodal image retrievals.

References

[1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional s paces. *Proceedings of ICDT*, 2001.

[2] E. L. Allwein, R. E. Schapire, and Y. Singer. Reducing multiclass to binary: A unifying approach for margin classifiers. *Journal of Machine Learning Research*, 1, 2000.

[3] K. Barnard and D. Forsyth. Learning the semantics of words and pictures. In *International Conference on Computer Vision*, volume 2, pages 408–415, 2000.

[4] K. Barnard and D. Forsyth. Clustering art. In *CVPR*, 2001.

[5] A. B. Benitez and S.-F. Chang. Semantic knowledge construction from annotated image collection. *Proceedings of IEEE International Conference on Multimedia*, 2002.

[6] L. Breiman. Bagging predicators. *Machine Learning*, pages 123–140, 1996.

[7] L. Breiman. Arcing classifiers. *The Annals of Statistics*, pages 801–849, 1997.

[8] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[9] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.pdf*, November 2000.

[10] E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[11] S.-F. Chang, W. Chen, and H. Sundaram. Semantic visual templates: Linking visual features to semantics. *Proceedings of IEEE International Conference on Image Processing*, 1998.

[12] O. Chapelle, V. Vapnik, O. Bousquet, and S. N. Mukherjee. Choosing multiple parameters for support vector machines. *International Conference in Machine Learning*, 2002.

[13] C. Cortes and V. Vapnik. Support vector networks. *Machine Learning*, 20:1–25, 1995.

[14] T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2, 1995.

[15] J. Friedman. Another approach to polychotomous classification. *Stanford University Technical Report*, 1996.

[16] K. Goh, E. Chang, and K. T. Cheng. Svm binary classifier ensembles for image classification. In *Proc. of ACM CIKM 2001*, Nov. 2001.

[17] K. Goh, B. Li, and E. Chang. Dyndex — an dynamic and non-metric space indexer. *ACM International Conference on Multimedia*, December 2002.

[18] E. B. Goldstein. *Sensation and Perception (5th Edition)*. Brooks/Cole, 1999.

[19] R. Herbrich, T. Graepel, and C. Campbell. Bayes point machines: Estimating the bayes point in kernel space. In *International Joint Conference on Artificial Intelligence Workshop on Support Vector Machines*, pages 23–27, 1999.

[20] R. Herbrich, T. Graepel, and C. Campbell. Bayes point machines. In *Journal of Machine Learning Research 1*, pages 245–279, 2001.

[21] A. Hinneburg, C. C. Aggarwal, and D. A. Keim. What is the nearest neighbor in high dimensional spaces? *Proceedings of the 26th VLDB*, September 2000.

[22] K. A. Hua, K. Vu, and J.-H. Oh. Sammatch: A flexible and efficient sampling-based image retrieval technique for image databases. *Proceedings of ACM Multimedia*, November 1999.

[23] T. Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 200–209. Morgan Kaufmann, 1999.

[24] J.-G. Leu. Computing a shape's moments from its boundary. *Pattern Recognition*, pages Vol.24, No.10,pp.949–957, 1991.

[25] B. Li and E. Chang. Discovery of a perceptual distance function for measuring image similarity. *ACM Multimedia Journal Special Issue (to appear)*, 2002.

[26] B. Li, E. Chang, and C.-T. Wu. Dynamic partial function. *IEEE Conference in Image Processing*, September 2002.

[27] B. Li, W.-C. Lai, E. Chang, and K.-T. Cheng. Minig image features for efficient query processing. *Proceedings of IEEE Data Mining*, November 2001.

[28] L. Luntz and V. Brailovsky. On estimation of characters obtained in statistical procedure in re cognition. *Technicheskaya Kibernetica (in Russian)*, 1969.

[29] W. Y. Ma and H. Zhang. Benchmarking of image features for content-based retrieval. *Proceedings of Asilomar Conference on Signal, Systems & Computers*, 1998.

[30] B. Manjunath, P. Wu, S. Newsam, and H. Shin. A texture descriptor for browsing and similarity retrieval. *Signal Processing Image Communication*, 2001.

[31] M. Moreira and E. Mayoraz. Improving pairwise coupling classification with error correcting classifiers. *Proceedings of the Tenth European Conference on Machine Learning*, April 1998.

[32] Y. Mori, H. Takahashi, and R. Oka. Automatic words assignment to images based on image division and vector quantization. In *Proc. of RIAO 2000: Content-Based Multimedia Information Access*, Apr. 2000.

[33] M. R. Naphade and T. S. Huang. A probabilistic framework for smeantic indexing and retrieval in video. *Proceedings of IEEE International Conference on Multimedia*, 2000.

[34] M. R. Naphade, C.-Y. Lin, J. Smith, B. Tseng, and S. Basu. Learning to annotate video databases. *SPIE Electronic Imaging 2002 - Storage and Retrieval for Media Databases*, January 2002.

[35] M. R. Naphade, C.-Y. Lin, and J. R. Smith. Learning semantic multimedia representations from a small set of examples. *Proceedings of IEEE International Conference on Multimedia*, 2002.

[36] S. Paek, C. Sable, V. Hatzivassiloglou, A. Jaimes, B. Schiffman, S. Chang, and K. McKeown. Integration of visual and text-based approaches for the content labeling and classification of photographs. *Workshop on Multimedia Indexing and Retrieval*, 1999.

[37] R. W. Picard and T. P. Minka. Vision texture for annotation. *Journal of Multimedia Systems*, 3:3–14, 1995.

[38] P. Rujan. Playing billard in version space. *Neural Computation*, 1997.

[39] P. Rujan and M. Marchand. Computing the bayes kernel classifier. *Advances in Large Margin Classifiers*, 2000.

[40] E. Saber and A. M. Tekalp. Region-based affine shape matching for automatic image annotation and query-by-example. *Journal of Visual Communication and Image Representation*, 8(1):3–20, Mar. 1997.

[41] R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.

[42] H. T. Shen, B. C. Ooi, and K.-L. Tan. Giving meaning to www images. *ACM International Conference on Multimedia*, 2000.

[43] J. Smith and S.-F. Chang. Automated image retrieval using color and texture. *IEEE Transaction on Pattern Analysis and Machine Intelligence*, November 1996.

[44] G. Sychay, E. Chang, and K.-S. Goh. Effective image annotation via active learning. *Proceedings of IEEE International Conference on Multimedia*, 2002.
[45] H. Tamura, S. Mori, and T. Yamawaki. Texture features corresponding to visual perception. *IEEE Transaction on Systems Man Cybernet (SMC)*, 1978.
[46] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *ACM International Conference on Multimedia*, October 2001.
[47] G. Valentini. Upper bounds on the training error of ecoc svm ensembles. *Technical Report*, Feb. 2001.
[48] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.
[49] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.
[50] J. Wang, J. Li, and G. Wiederhold. Simplicity: Semantics-sensitive integrated matching for picture libraries. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(9), 2001.
[51] T. Watkin. Optimal learning with a neural network. *Europhysics Letters*, 1993.
[52] L. Wenyin, S. Dumais, Y. Sun, H. Zhang, M. Czerwinski, and B. Field. Semi-automatic image annotation. In *Proc. of Interact 2001: Conference on Human-Computer Interaction*, pages 326–333, July 2001.

Appendix A

pattern	500	texture	500	frog	99	turtle	79	bear	187
buffalo	56	camel	46	cat	137	cattle	95	cheetah	44
chicken	45	deer	319	dog	551	elephant	178	fox	114
giraffe	60	goat	54	hippopotamus	66	horse	94	leopard	87
lion	149	monkey	177	pig	49	rabbit	53	ram	103
rhinoceros	98	sheep	48	squirrel	55	tiger	145	wildcat	276
wolf	184	zebra	77	bridge	393	building	683	castle	477
church	372	lighthouse	196	palace	398	pyramid	80	ruins	500
skyscraper	283	temple	317	tower	205	windmill	47	crane	180
duck	193	eagle	177	ostrich	18	owl	135	parrot	98
penguin	191	apple	76	berry	93	beverage	151	cake	71
dish	400	orange	41	seed	147	vegetable	125	bug	207
butterfly	350	canyon	164	cave	146	city	500	cliff	156
coast	600	desert	619	field	600	forest	500	garden	600
industry	335	island	69	lake	518	mountain	700	rock	315
snow	648	waterfall	526	cloud	159	firework	223	sky	294
space	107	sunset	500	wave	171	bead	107	china	105
furniture	146	machine	138	instrument	109	precious stone	222	seashell	99
ship	95	sign	127	tool	304	train	86	child	83
couple	115	group	57	man	69	woman	457	bonsai	101
cactus	134	grass	44	leaf	311	mushroom	221	rose	121
tree	325	tulip	71	wildflower	301	alligator	101	lizard	244
snake	172	dolphin	55	fish	506	seal	64	shark	25
whale	106								

Table 5: Category Names and Sizes for the 25k-image Dataset.